如何部署 Milvus

> **文档编号**: doc_002
> **主题**: Milvus 向量数据库的多种部署方式与详细操作指南
> **适用场景**: RAG 知识库检索、向量数据库运维、AI 应用开发环境搭建

一、Milvus 部署模式概览

Milvus 提供三种主要部署模式，分别适用于不同规模和场景： cite web_search:1#4

部署模式	适用场景	数据规模	推荐度
Milvus Lite	本地开发、Jupyter Notebook、边缘设备	< 100 万向量	⭐ 开发测试
Milvus Standalone	单机生产环境、中小规模应用	百万 ~ 千万级	强烈推荐
Milvus Distributed	分布式集群、大规模生产环境	亿级以上	生产必选

二、Milvus Lite 部署（开发测试）

Milvus Lite 是 Milvus 的轻量级版本，已包含在 Python SDK 中，适合快速原型验证。 cite web_search:1#4

2.1 环境要求

- Ubuntu >= 20.04（x86_64 和 arm64）
- MacOS >= 11.0（Apple Silicon M1/M2 和 x86_64）
- **注意：尚不支持 Windows**

2.2 安装与启动

```
# 安装 pymilvus（已包含 Milvus Lite）
pip install pymilvus
```

```
# Python 代码中直接启动
from milvus import default_server

# 启动服务
default_server.start()

# 使用 MilvusClient 连接
from pymilvus import MilvusClient
client = MilvusClient("milvus_demo.db")  # 本地 SQLite 文件存储

# 停止服务
# default_server.stop()
```

2.3 特点说明

- **零配置启动**：无需 Docker 或 K8s，pip 安装即用

- **API 一致性**：与 Standalone/Distributed 使用相同 API
- **局限性**：仅适用于小规模原型设计（通常少于 100 万向量）

三、Milvus Standalone 部署（推荐方式）

Docker Compose 方式是单机部署的**推荐方案**，包含 Milvus 服务、etcd（元数据存储）和 MinIO（对象存储）三个核心组件。 cite web_search:1#4 web_search:1#10

3.1 系统要求

- **CPU**： 8 核
- **内存**： 16GB（向量检索依赖内存）
- **磁盘**： 100GB（SSD 优先）
- **Docker**：已安装 Docker 和 Docker Compose

3.2 前置准备：安装 Docker

```
# 更新系统
sudo apt update && sudo apt upgrade -y

# 安装必要依赖
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release

# 添加阿里云 Docker 镜像源（国内环境推荐）
curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/
↪  keyrings/docker-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
↪  https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/
↪  sources.list.d/docker.list > /dev/null

# 安装 Docker
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# 启动 Docker
sudo systemctl start docker
sudo systemctl enable docker
```

3.3 配置镜像加速（国内环境必需）

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json <<-'EOF'
{
  "registry-mirrors": [
    "https://< 你的阿里云镜像加速器地址 >.mirror.aliyuncs.com",
    "https://docker.mirrors.ustc.edu.cn",
    "https://hub-mirror.c.163.com",
    "https://swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io"
  ]
```

```
}
EOF

sudo systemctl daemon-reload
sudo systemctl restart docker
```

> 💡 **获取阿里云镜像加速器**：登录阿里云控制台 → 容器镜像服务 → 镜像加速器

3.4 下载并配置 Docker Compose 文件

```
# 创建工作目录
mkdir -p ~/milvus && cd ~/milvus

# 下载官方 docker-compose.yml（以 v2.5.21 为例）
wget https://github.com/milvus-io/milvus/releases/download/v2.5.21/milvus-standalone-docker-compose.yml
 ↪  -O docker-compose.yml

# 创建数据持久化目录
mkdir -p volumes/milvus
```

3.5 启动 Milvus 服务

```
# 拉取镜像（国内环境可能需要多次尝试）
sudo docker compose pull

# 启动服务（后台运行）
sudo docker compose up -d

# 查看运行状态
sudo docker compose ps

# 查看日志
sudo docker logs milvus-standalone -f
```

镜像拉取失败的替代方案

```
# 若 minio 镜像拉取失败，从 Quay.io 拉取
docker pull quay.io/minio/minio:RELEASE.2024-05-28T17-19-04Z
docker tag quay.io/minio/minio:RELEASE.2024-05-28T17-19-04Z minio/minio:RELEASE.2024-05-28T17-19-04Z

# 若 milvus 镜像拉取失败，从华为云镜像拉取
docker pull swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/milvusdb/milvus:v2.5.21
docker tag swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/milvusdb/milvus:v2.5.21 docker.io/
 ↪  milvusdb/milvus:v2.5.21
```

3.6 验证部署成功

```
# 1. 查看容器状态（Up 状态即为成功）
docker ps | grep milvus-standalone

# 2. 使用 Python SDK 验证连接
```

```
python -c "
from pymilvus import MilvusClient
client = MilvusClient(uri='http://localhost:19530')
print('Successfully connected to Milvus!')
print('Version:', client.get_server_version())
"
```

3.7 启用账号密码认证（可选，生产环境推荐）

步骤 1：创建用户配置文件

```
cat > ~/milvus/user.yaml << 'EOF'
common:
  security:
    authorizationEnabled: true
EOF
```

步骤 2：修改 docker-compose.yml 在 `standalone.volumes` 下新增挂载项：

```
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
      - ${DOCKER_VOLUME_DIRECTORY:-.}/user.yaml:/milvus/configs/user.yaml
```

步骤 3：重启服务

```
docker compose down
docker compose up -d
```

步骤 4：使用认证连接

```
from pymilvus import MilvusClient

# 方式 1: 使用 token（推荐）
client = MilvusClient(
    uri="http://localhost:19530",
    token="root:Milvus"  # 默认用户名: root, 默认密码: Milvus
)

# 方式 2: 使用 user/password 参数
client = MilvusClient(
    uri="http://localhost:19530",
    user="root",
    password="Milvus"
)
```

修改默认密码

```
from pymilvus import connections, utility

connections.connect(
    alias='default',
```

```
    host='localhost',
    port='19530',
    user='root',
    password='Milvus'
)

# 修改 root 密码
utility.reset_password('root', 'Milvus', 'YourNewStrongPassword', using='default')
```

四、Milvus Distributed 部署（生产集群）

基于 Kubernetes + Helm 的分布式部署适用于大规模生产环境。 cite web_search:1#4

4.1 前置要求

- Kubernetes 集群（ 3 节点）
- kubectl 和 Helm 3 已安装
- 持久化存储（PVC）已配置

4.2 安装步骤

```
# 安装 kubectl
curl -LO "https://dl.k8s/release/$(curl -L -s https://dl.k8s/release/stable.txt)/bin/linux/amd64/
↪ kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# 安装 Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# 添加 Milvus Helm 仓库
helm repo add milvus https://zilliztech.github.io/milvus-helm/
helm repo update

# 安装 Milvus（单机模拟集群，适合测试）
helm install my-milvus milvus/milvus --set cluster.enabled=false --set etcd.replicaCount=1 --set
↪ minio.mode=standalone --set image.repository=registry.cn-hangzhou.aliyuncs.com/milvusdb/milvus

# 安装 Milvus（真正分布式集群）
helm install my-milvus milvus/milvus --set cluster.enabled=true --set etcd.replicaCount=3 --set
↪ minio.mode=distributed --set queryNode.replicas=2 --set dataNode.replicas=2
```

4.3 生产环境资源配置建议

组件	CPU	内存	磁盘	说明
协调器（etcd）	2 核	4GB	50GB	元数据管理，必须高可用
数据节点	8 核	16GB	1TB	存储向量数据

Table 2 – continued

组件	CPU	内存	磁盘	说明
查询节点	8 核	32GB	-	检索依赖内存，重点配置
代理节点	4 核	8GB	-	接收客户端请求
索引节点	8 核	16GB	-	构建向量索引

五、WEB 管理工具：Attu

Attu 是 Milvus 的官方图形化管理工具，提供可视化界面管理集合、数据、索引等。 cite web_search:1#4

5.1 Docker 方式安装 Attu

```
# 下载 Attu 的 docker-compose 文件
wget https://github.com/zilliztech/attu/releases/download/v2.4.5/attu-docker-compose.yml -O
↪  attu-docker-compose.yml

# 启动 Attu
docker compose -f attu-docker-compose.yml up -d

# 访问 http://localhost:8000 打开管理界面
```

5.2 连接配置

- **Milvus 地址**: `localhost:19530`（或实际服务器 IP）
- **认证信息**: 如启用认证，填写 `root:Milvus`

六、常见问题与解决方案

问题现象	可能原因	解决方案
镜像拉取失败	网络限制或镜像源问题	配置 Docker 镜像加速器，或手动拉取替代镜像后重新打标签
端口 19530 被占用	其他服务占用端口	`sudo lsof -i :19530` 查看占用进程，或修改 docker-compose.yml 端口映射
内存不足导致 OOM	系统内存 < 8GB	增加物理内存，或修改 docker-compose.yml 限制资源使用
防火墙无法连接	端口未开放	`sudo ufw allow 19530/tcp` 开放端口
认证连接失败	用户名或密码错误	确认使用 `root:Milvus`，注意大小写
集合加载失败	未执行 load_collection	检索前必须调用 `client.load_collection()` 将数据加载到内存

七、性能优化建议

7.1 索引优化

```
# 创建 HNSW 索引（平衡速度与精度，推荐）
client.create_index(
    collection_name="my_collection",
    index_params={
        "index_type": "HNSW",
        "metric_type": "COSINE",
        "params": {"M": 16, "efConstruction": 200}
    }
)
```

7.2 批量操作

- **插入**：单次批量 1000~10000 条，减少网络开销
- **检索**：支持批量查询向量，提升吞吐量

7.3 向量压缩（亿级数据）

大规模场景使用 PQ（Product Quantization）量化，存储成本可降低 70%：

```
index_params = {
    "index_type": "PQ",
    "metric_type": "COSINE",
    "params": {"nlist": 128, "m": 16, "nbits": 8}
}
```

7.4 监控告警

集成 Prometheus + Grafana 监控 Milvus 指标：

1. 启用 Milvus 监控端口（默认 9091）
2. 配置 Prometheus 抓取 Milvus 指标
3. 导入 Milvus 官方 Grafana 仪表盘（ID: 17315），监控 QPS、延迟、内存占用等

八、部署方式选择决策树

```
开始部署 Milvus
    │
    ├─ 数据量 < 100 万向量?
    │    ├─ 是 → 使用 Milvus Lite (pip install pymilvus)
    │    └─ 否 → 继续判断
    │
    ├─ 需要高可用/分布式扩展?
    │    ├─ 是 → 使用 Milvus Distributed (K8s + Helm)
    │    └─ 否 → 继续判断
    │
    └─ 使用 Milvus Standalone (Docker Compose) ← 推荐
```

九、总结

Milvus 的部署方式灵活多样：

- **开发阶段**：使用 **Milvus Lite**，5 分钟快速启动，零配置
- **生产单机**：使用 **Milvus Standalone + Docker Compose**，稳定可靠，运维简单
- **大规模生产**：使用 **Milvus Distributed + Kubernetes**，支持水平扩展和高可用

无论选择哪种方式，建议始终启用**数据持久化**和**定期备份**，并根据实际数据规模选择合适的**索引类型**和**硬件资源配置**。

相关文档：doc_001（CANN 核心优势）
关键词：Milvus, 向量数据库, Docker, Kubernetes, Helm, 部署指南, RAG, 向量检索, Attu